Policy Gradients in Two Days

A hidden-code briefing: REINFORCE on CartPole, then variance reduction
grounded in Lambert, *Reinforcement Learning from Human Feedback* (rlhfbook.com, Aug. 2026)

Briefing and tests by Claude. Every line of `pg.py` is yours.

September 2026

How this document works

Same format as the ray tracer. Each part has a **lecture** that teaches the theory completely, so you never need to look anything up; a **problem** stated like an exam question with all givens; **acceptance criteria**; and **debugging signatures**. What it never contains: implementation code, data structures, or step sequences. Turning the lecture into a program is the problem set.

The theory follows Nathan Lambert's *Reinforcement Learning from Human Feedback* (rlhfbook.com, August 2026 build), Chapter 3 for the problem formulation and Chapter 6 for the algorithms. His prose is better than a paraphrase of it, so where he has said the thing, his words are used and a footnote gives the section, so the text reads straight through and you can open the original beside it. The derivations and proofs are the parts he leaves to the reader; those are mine. His book is about language models; this briefing runs the same mathematics on CartPole, which he uses as his own worked example in §3.1.2. Where the two settings differ, he says how, and he is quoted on that too.

The tests in Section 10 fix a small interface: a handful of function names with their inputs and outputs. That contract is the only constraint on your design.

Givens for both days. Python, `torch`, `gymnasium`. The environment is `CartPole-v1`. In Lambert's notation (§3.1.2, eq. 4) the state is $s_t = (x_t, \dot{x}_t, \theta_t, \dot{\theta}_t)$: cart position and velocity, pole angle and angular velocity. Two actions, push left or right. Reward $r_t = 1$ every step the pole stays within 12° and the cart within ±2.4; the episode *terminates* when either bound is violated, and is *truncated* at 500 steps. Both end the episode. Maximum return is 500. The `gymnasium` API: `obs, info = env.reset(seed=s)` and `obs, r, terminated, truncated, info = env.step(a)`.

In RL, an agent takes actions a_t sampled from a policy $\pi(a_t \mid s_t)$ given the state of the environment s_t to maximize reward $r(s_t, a_t)$. A policy is a function that maps each state to a probability distribution over actions. The early policies that evolved into modern literature on RLHF were in what is called deep reinforcement learning – when a neural network is used to learn said function. Traditionally, the environment evolves according to transition (dynamics) $p(s_{t+1} \mid s_t, a_t)$ with an initial state distribution $\rho_0(s_0)$. Together, the policy and dynamics induce a trajectory distribution.[1]

Initially, the thermostat's policy is essentially random – it flips the heater on and off with no regard for the current temperature, and the room's temperature swings wildly. Over many episodes of trial and error, the agent discovers that turning the heater on when the room is cold

[1] Lambert, §3.1.

and off when it is warm leads to more reward, and gradually converges on a sensible policy. This is the core RL loop: observe a state, choose an action, receive a reward, and update the policy to get more reward over time.[2]

His §3.1.2 then writes out CartPole's dynamics as explicit Euler integration (eqs. 5–9), which is essentially what `gymnasium` runs. Keep that in mind for one point below: the environment is a black box *to autograd*, not a mystery. You could differentiate it by hand; the method simply never needs to.

Day 1 — REINFORCE

1 Lecture: the setting

1.1 Policy, trajectory, return

An agent observes a state s_t, samples an action a_t from its **policy** $\pi_\theta(a \mid s)$, receives reward r_t, and the environment moves to s_{t+1} according to its dynamics $p(s_{t+1} \mid s_t, a_t)$. The policy is a probability distribution over actions given the state, with parameters θ: here, the weights of a small network mapping four numbers to two logits, softmaxed. The agent *samples* from those probabilities rather than taking the argmax. That randomness is what the method runs on.

One episode is a **trajectory** $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$. Its **return from time** t is the discounted sum of rewards from that point on:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k \geq 0} \gamma^k r_{t+k}, \qquad G_t = r_t + \gamma G_{t+1}.$$

The recursive form is how you will compute it. The total return of the trajectory is $R(\tau) = G_0$.[3]

1.2 The objective

We want the policy that collects the most reward in expectation:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\big[R(\tau)\big].$$

Trajectories are random because the policy samples actions and, in general, because the environment may be stochastic. $p_\theta(\tau)$ is the probability of a whole trajectory under the current policy. We want $\nabla_\theta J$ so we can climb it: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.[4]

1.3 Why you cannot simply backpropagate

In supervised learning the loss is a differentiable function of the network output. Here it is not. The reward comes from a physics step that autograd never sees, and θ never touches r_t directly. It influences reward only by changing *which trajectories are likely*. The dependence on θ lives inside the distribution p_θ, not inside the quantity being averaged. The next section is the trick that moves it out.

[2] Lambert, §3.1.1, the thermostat.

[3] Lambert §6.2, eqs. 25–26 define G_t exactly this way; eq. 27 defines the value function $V(s) = \mathbb{E}[G_t \mid s_t = s]$, which returns on Day 2.

[4] Lambert §6.2, eqs. 28–32. He writes $J(\theta) = \sum_s d_0(s) V^{\pi_\theta}(s)$ over initial states, then the sampled form $\hat{J} = \frac{1}{B}\sum_i R(\tau_i)$, then eq. 32, $J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[R(\tau)]$, which is where we start.

2 Lecture: deriving the policy gradient

2.1 The log-derivative trick

Write the objective as an integral over trajectories and take the gradient. $R(\tau)$ has no θ in it, so the gradient falls on p_θ alone:

$$J(\theta) = \int p_\theta(\tau) R(\tau)\, d\tau, \qquad \nabla_\theta J = \int \nabla_\theta p_\theta(\tau)\, R(\tau)\, d\tau.$$

This is not an expectation, because $\nabla_\theta p_\theta$ is not a density, so it cannot be estimated by sampling. One identity fixes it. By the chain rule, $\nabla_\theta \log p = \nabla_\theta p / p$ for any positive p, so

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau).$$

Substitute, and p_θ returns as a weight:

$$\nabla_\theta J = \int p_\theta(\tau)\, R(\tau)\, \nabla_\theta \log p_\theta(\tau)\, d\tau = \mathbb{E}_{\tau \sim p_\theta}\big[R(\tau)\, \nabla_\theta \log p_\theta(\tau)\big].$$

It is an expectation again, and expectations are estimated by sampling: run episodes, compute the bracket for each, average. This is the whole conceptual leap. What follows is bookkeeping.[5]

2.2 Which part of the trajectory depends on θ

A trajectory's probability is the product of everything that happened, in order:

$$p_\theta(\tau) = \rho_0(s_0) \prod_t \pi_\theta(a_t \mid s_t)\, p(s_{t+1} \mid s_t, a_t),$$

with ρ_0 the initial-state distribution and p the dynamics. Take the log; the product becomes a sum. Take the gradient with respect to θ: the initial-state term does not involve θ, gone; the dynamics terms do not involve θ, gone. Only the policy survives:

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

Pause on it. The physics dropped out of the gradient entirely. You do not model CartPole, know its equations, or differentiate through it. You need only the gradient of your own network's log-probability of the actions it actually took. This is what *model-free* means.

Reaching this equation is a crucial point in the implementation. Here, we have gone far enough to see that the gradient of the trajectory distribution reduces to a sum of gradients from language model policy probabilities (which are just the probabilities of tokens given by the model we're training). In practice, this results in a common form of the policy gradient equations. They end up looking like a sum of log-probabilities in the loss, and then we compute the gradients via autodiff.[6]

For you the "language model policy probabilities" are the two softmaxed logits of a four-input MLP. Nothing else changes.

2.3 The estimator and the general form

Combine the two results:

$$\nabla_\theta J = \mathbb{E}_{\tau \sim p_\theta}\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)\Big].$$

[5] Lambert §6.2.1, eqs. 33–37. Same three lines, same name for the trick.

[6] Lambert, §6.2.1, after eq. 39.

In words: for each action taken, push up its log-probability, scaled by how good the whole episode was. Good episodes make all their actions more likely; bad episodes, less. From N sampled episodes the Monte Carlo estimate is

$$\hat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t} \nabla_\theta \log \pi_\theta\big(a_t^{(i)} \mid s_t^{(i)}\big)\, R(\tau^{(i)}),$$

unbiased for $\nabla_\theta J$, and noisy. Everything in Day 2 keeps the shape and changes the weight. Lambert writes that shape once and for all as

$$g = \nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\Big[\sum_{t} \Psi_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big],$$

where Ψ_t is whatever number multiplies the log-probability gradient at step t. Day 1 uses $\Psi_t = R(\tau)$. Day 2 walks down his list.

Quite often, people use a more general formulation of the policy gradient [$g = \nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[\sum_t \Psi_t \nabla_\theta \log \pi_\theta(a_t|s_t)]$]. Where Ψ_t can be the following (where the rewards can also often be discounted by γ), a taxonomy adopted from Schulman et al. 2015: 1. $R(\tau) = \sum_t r_t$: total reward of the trajectory. 2. $\sum_{t' \geq t} r_{t'}$: reward following action a_t, also described as the return from time t, G_t. 3. $\sum_{t' \geq t} r_{t'} - b(s_t)$: baselined version of previous formula. 4. $Q^\pi(s_t, a_t)$: state-action value function. 5. $A^\pi(s_t, a_t)$: advantage function, which yields the lowest possible theoretical variance if it can be computed accurately. 6. $r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)$: Temporal Difference (TD) residual.[7]

This briefing implements items 1, 2, and 3. Day 1 is item 1.

2.4 From gradient to loss

Autograd minimizes scalars. We want to *ascend* J. Define a surrogate whose gradient is $-\hat{g}$:

$$L(\theta) = -\frac{1}{N} \sum_{i} \sum_{t} \log \pi_\theta\big(a_t^{(i)} \mid s_t^{(i)}\big) \cdot \Psi_t^{(i)},$$

with the weights Ψ **treated as constants**: numbers computed from rewards, carrying no gradient. One descent step on L is one ascent step on J. Two warnings. The *value* of L means nothing; it is not a performance measure and will not decrease monotonically. And Ψ must never be differentiated through, or on Day 2 you will train the value network through the policy loss.

In order to understand this equation, it is good to understand different cases that can fall within a batch of updates. Remember that we want the loss to decrease as the model gets better at the task. Case 1: Positive advantage, so the action was better than the expected value of the state. We want to reinforce this. In this case, the model will make this more likely with the negative sign. To do so, it'll increase the logratio. A positive logratio, or sum of log probabilities of the tokens, means that the model is more likely to generate those tokens. Case 2: Negative advantage, so the action was worse than the expected value of the state. This follows very similarly. Here, the loss will be positive if the new model was more likely, so the model will try to make it so the policy parameters make this completion less likely. Case 3: Zero advantage, so no update is needed. The loss is zero, don't change the policy model.[8]

His `ratio` is $\pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$, which is identically 1 when you take one gradient step per batch, and then $\nabla_\theta(\text{advantage} \times \text{ratio}) = \text{advantage} \times \nabla_\theta \log \pi_\theta$, our form. The ratio earns its keep only when you reuse data across several steps, which is PPO's business, not this week's.

[7] Lambert, §6.2.1, eq. 41 and following.

[8] Lambert, §6.3.1, on `pg_loss = -advantages * ratio`.

2.5 The update loop

Collect a batch of episodes with the current policy, recording states, actions, rewards. Compute Ψ from the rewards. Compute L. One optimizer step. Discard the data and repeat. Discard, because the estimator is unbiased only for samples from the *current* policy; after the step, the data is from the old one.

The default implementation for policy-gradient algorithms is what is called on-policy execution, where the actions (generations) taken by the agent (language model) are scored before updating the model. The theoretical derivations of policy-gradient rely on all actions being exactly on-policy where the model is always up to date with the results from the latest trials/rollouts. In practice, maintaining exact on-policy execution substantially slows training – and perfect synchronization is technically impossible regardless.[9]

You are in the clean regime: one update per batch, then fresh data. The corrections his systems need do not apply.

3 Problem 1: REINFORCE on CartPole

Build. A policy network from the 4-dimensional observation to 2 logits. A function giving each step of a finished episode its weight (Day 1: the episode's total return, identical for every step). A surrogate loss taking logits, actions, and weights. A training loop that collects episodes in batches, updates, and records each episode's undiscounted return. Sample actions from the policy during collection.

Givens. MLP, one hidden layer of 64 units. Adam, learning rate 10^{-2}. Batch of 10 episodes per update. $\gamma = 0.99$. 1500 episodes, seed 0.

Acceptance. Per-episode return climbs from about 20 (random policy) toward 500, noisily. Criterion: some window of 20 consecutive episodes averages at least 195. That number is CartPole-v0's registered reward threshold; the classic "solved" convention asks for it over 100 consecutive episodes, and the 20-episode window is a deliberate relaxation for a vanilla estimator. Do not expect a stable 500 today.

4 Debugging signatures, Day 1

- **Flat at about 20 forever.** No learning. Loss has the wrong sign; or actions come from argmax so exploration is zero; or log-probabilities were computed from detached tensors so no gradient reaches the policy.
- **Climbs, then collapses to 20 and stays.** A near-deterministic policy took a bad step into a corner it cannot sample out of. Usually learning rate. Genuine REINFORCE behavior; Day 2 tames it.
- **NaN loss.** log of a zero probability. Use `log_softmax` or `Categorical.log_prob`, never log(softmax(·)).
- **Identical weights within an episode is correct on Day 1.** If they differ, you implemented reward-to-go early.
- **Episodes never reach 500.** You check `terminated` only; truncation also ends the episode.

[9] Lambert, §6.3.3.

Day 2 — Variance reduction

5 Lecture: why the estimator is noisy, and the identity that fixes it

Two sources of noise. First, every action is credited with the *entire* episode's return, including rewards paid out before the action happened, which it cannot have caused. Second, in CartPole every reward is $+1$, so $R(\tau) > 0$ always: every action in every episode gets pushed *up*, and the only signal is that longer episodes push harder. The learning lives in the difference between weights, but each weight is large, so the estimate swings batch to batch. Both fixes rest on one identity.

5.1 The score function has zero mean

For any state s, the expected gradient of the log-probability under the policy's own action distribution is zero:

$$\begin{aligned}\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\big[\nabla_\theta \log \pi_\theta(a \mid s)\big] &= \sum_a \pi_\theta(a \mid s)\, \nabla_\theta \log \pi_\theta(a \mid s) = \sum_a \nabla_\theta \pi_\theta(a \mid s) \\ &= \nabla_\theta \sum_a \pi_\theta(a \mid s) = \nabla_\theta 1 = 0.\end{aligned}$$

Step by step: write the expectation as a sum; apply $\pi\, \nabla_\theta \log \pi = \nabla_\theta \pi$, the log-derivative trick in reverse; swap sum and gradient, legal for a finite sum; probabilities sum to one; the gradient of a constant is zero. OpenAI's Spinning Up calls this the Expected Grad-Log-Prob (EGLP) lemma and builds the same two results on it.[10] One test checks it numerically on your network.

A common problem with vanilla policy gradient algorithms is the high variance in gradient updates, which can be mitigated in multiple ways. The high variance comes from the gradient updates being computed by estimating the return G from an often small set of rollouts in the environment that tend to be susceptible to noise (e.g. the stochastic nature of generating from language models with temperature > 0). The variance across return estimates is higher in domains with sparse rewards, as more of the samples are 0 or 1, rather than closely clustered. In order to alleviate this, various techniques are used to normalize the value estimation, called baselines. Baselines accomplish this in multiple ways, effectively normalizing by the value of the state relative to the downstream action (e.g. in the case of Advantage, which is the difference between the Q value and the value). The simplest baselines are averages over the batch of rewards or a moving average. Even these action-independent baselines can reduce variance without changing the expected gradient, since $\mathbb{E}_{a\sim\pi(a|s)}[b(s)\nabla_\theta \log \pi_\theta(a|s)] = 0$ for any state-dependent $b(s)$, improving the learning signal substantially.[11]

That last sentence is the theorem. The five-step chain above is the proof he leaves to the reader.

6 Lecture: reward-to-go

An action at step t cannot influence rewards before t, so drop them. Replace $\Psi_t = R(\tau)$ with $\Psi_t = G_t = \sum_{t'\geq t} \gamma^{t'-t} r_{t'}$. The claim is that the estimator stays unbiased.

Proof. Take one term of the Day 1 estimator and split $R(\tau)$ into rewards before and after t:

$$\mathbb{E}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)\big] = \mathbb{E}\Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{t'<t} \gamma^{t'} r_{t'}\Big] + \mathbb{E}\Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{t'\geq t} \gamma^{t'} r_{t'}\Big].$$

[10] Spinning Up, *Intro to Policy Optimization*, Part 3.
[11] Lambert, §6.2.2.

Fix a past reward $r_{t'}$, $t' < t$. Condition on the history up to and including s_t. Given that history, $r_{t'}$ is a fixed number, already paid; the only remaining randomness is $a_t \sim \pi_\theta(\cdot \mid s_t)$. So

$$\mathbb{E}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, r_{t'}\big] = \mathbb{E}_{\text{hist}}\Big[r_{t'}\, \mathbb{E}_{a_t}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\big]\Big] = \mathbb{E}_{\text{hist}}\big[\, r_{t'} \cdot 0\,\big] = 0$$

by the identity. Every past reward vanishes; the future stays. Reward-to-go is exact.

Honest footnote. What the proof leaves is $\sum_{t' \geq t} \gamma^{t'} r_{t'} = \gamma^t G_t$, with a leading γ^t; Sutton and Barto's REINFORCE keeps it.[12] Standard practice drops it and uses G_t: exact when $\gamma = 1$, and otherwise a small deliberate bias in exchange for not down-weighting late actions. Know that you are doing it. Appendix 11 confirms both halves of that sentence by exact enumeration.[13]

7 Lecture: baselines, two ways

7.1 The theorem

Subtracting from Ψ_t any quantity b that does not depend on the action a_t leaves the estimator unbiased:

$$\mathbb{E}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\,(G_t - b)\big] = \mathbb{E}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\big].$$

Proof. Condition on everything except a_t. Then b is a constant and the subtracted term is $\mathbb{E}[\, b \cdot \mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)]\,] = 0$. Same identity, one line. The restriction is the whole content: b may depend on the current state, on anything already determined before a_t was drawn, or on other episodes entirely, but not on a_t or on anything downstream of it, such as later rewards in the same episode. Otherwise it does not factor out of the inner expectation, and the estimator is biased.

This gives a generic algorithm, REINFORCE proper:

$$\nabla_\theta J = \mathbb{E}\Big[\sum_t \big(G_t - b(s_t)\big) \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big],$$

and the baselined weight $G_t - b(s_t)$ is called the **advantage** A_t: how much better this action did than expected. Positive for above-average actions, negative for below-average, and now bad actions in good episodes get pushed *down*, which the raw return could never do.

The algorithm REINFORCE is likely a backronym, but the components of the algorithm it represents are quite relevant for modern reinforcement learning algorithms. Defined in the seminal paper *Simple statistical gradient-following algorithms for connectionist reinforcement learning*: "The name is an acronym for 'REward Increment = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility.'" The three components of this are how to do the reward increment, a.k.a. the policy gradient step. It has three pieces to the update rule: 1. Nonnegative factor: This is the learning rate (step size) that must be a positive number, e.g. α below. 2. Offset Reinforcement: This is a baseline b or other normalizing factor of the reward to improve stability. 3. Characteristic Eligibility: This attributes the scalar reward signal to the parameters that produced the action. Williams denotes this eligibility term as e (not the exponential function). In modern policy-gradient notation, it corresponds to $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$. Thus, the form looks quite familiar: $\Delta\theta = \alpha(r - b)e$.[14]

There are two standard ways to get a b. They correspond to two ways of framing the problem, and you will implement both.

[12] Sutton and Barto, *Reinforcement Learning*, 2nd ed., §13.3, where the update is $\theta \leftarrow \theta + \alpha\, \gamma^t G_t \nabla_\theta \ln \pi$.

[13] Lambert §6.2.1, taxonomy item 2, and §6.2.2, eq. 43: $\nabla_\theta J = \mathbb{E}[\sum_t G_t \nabla_\theta \log \pi_\theta(a_t|s_t)]$, his "vanilla policy gradient" with respect to the time-t return.

[14] Lambert, §6.2.3.

7.2 Way one: a learned value function (MDP framing)

The ideal baseline is what the current policy expects from this state, the state-value function $V^\pi(s) = \mathbb{E}[G_t \mid s_t = s]$. Then $A_t = G_t - V^\pi(s_t)$, and each step gets its own advantage. V^π is unknown, so fit $V_\phi(s)$, a second small network. The observed G_t is an unbiased sample of $V^\pi(s_t)$, so this is ordinary regression: minimize $\frac{1}{M}\sum(V_\phi(s_t) - G_t)^2$ over the batch. Two networks, two optimizers, two losses, both updated once per batch. And once more: when A_t becomes a policy weight it must be a constant. No gradient from the policy loss into ϕ.

The value function within PPO is an additional copy of the model that is used to predict the value per token. The value of a token (or state) in traditional RL is predicting the future return from that moment, often with discounting. This value in PPO is used as a learned baseline, representing an evolution of the simple Monte Carlo version used with REINFORCE (which doesn't need the learned value network). This highlights how PPO is an evolution of REINFORCE and vanilla policy-gradient in multiple forms, across the optimization form, baseline, etc.[15]

His reference implementation in the same section computes returns backward as $G_t = r_t + \gamma(1 - \text{done}_t)\, G_{t+1}$, regresses V onto them with MSE, and ends with `advantages = (targets - v_pred).detach()`. That `.detach()` is the constant-weight rule.

7.3 Way two: leave-one-out over the batch (bandit framing)

No value network. Treat each episode as a single action with a single scalar reward R_i, and use the *other* episodes in the batch as the baseline. For a batch of K episodes,

$$b_i = \frac{1}{K-1}\sum_{j \neq i} R_j, \qquad A_i = R_i - b_i,$$

and every step of episode i receives the same weight A_i. This is REINFORCE Leave-One-Out. Unbiasedness follows from the theorem: b_i is built from other episodes' returns, which are independent of every action in episode i, so it factors out. Note the "leave one out" is load-bearing. If the baseline included R_i it would depend on episode i's own actions and the proof fails; the book makes exactly this point.

The core implementation detail of REINFORCE Leave One Out versus standard REINFORCE is that it takes the average reward of the other samples in the batch to compute the baseline – rather than averaging over all rewards in the batch. By excluding the current sample's reward from its own baseline, the RLOO baseline is independent of the action being evaluated, which keeps the gradient estimator exactly unbiased. Crucially, this only works when generating multiple trajectories (completions) per state (prompt), which is common practice in multiple domains of fine-tuning language models with RL.[16]

The MDP (token-level) view treats each token a_t as an action with state s_t being the running prefix. In practice, this is the framing used when we compute token-level advantages with a learned value function $V(s_t)$ (e.g., GAE) and apply KL penalties per token. PPO with a learned value network is the canonical example. In contrast, the bandit (sequence-level) view treats the whole completion as a single action with one scalar reward R. In code, this means computing a sequence-level advantage A_{seq} and broadcasting it to all tokens. RLOO and GRPO-style advantages are often used in this bandit-style setting.[17]

In his setting the K samples are K completions of one prompt; in yours they are K episodes from the same start distribution. Replace "token" with "step" and "completion" with "episode" and that paragraph describes your two Day 2 runs exactly.

[15] Lambert, §6.2.7.
[16] Lambert, §6.2.4.
[17] Lambert, §6.3.2.2, MDP vs. Bandit framing.

A small identity you can test. Subtracting the *full* batch mean, $R_i - \frac{1}{K}\sum_j R_j$, is not leave-one-out, but it is a constant multiple of it:

$$\begin{aligned}\frac{K}{K-1}\Big(R_i - \frac{1}{K}\sum_j R_j\Big) &= \frac{K}{K-1}R_i - \frac{1}{K-1}\sum_j R_j \\ &= \frac{K}{K-1}R_i - \frac{1}{K-1}R_i - \frac{1}{K-1}\sum_{j\neq i} R_j \\ &= R_i - \frac{1}{K-1}\sum_{j\neq i} R_j.\end{aligned}$$

Line two splits the sum into the i-th term and the rest; line three collects R_i. So mean-baselining is RLOO scaled by $\frac{K}{K-1}$, and a scale is absorbed by the learning rate. Both are unbiased in practice; one test checks the identity on your function.

Intuitively, the GRPO update is comparing multiple answers to a single question within a batch. The model learns to become more like the answers marked as correct and less like the others. This is a very simple way to compute the advantage, which is the measure of how much better a specific action is than the average at a given state. [...] As an example of how these algorithms are intertwined, we can show that the advantage estimation in a variant of GRPO, Dr. GRPO (GRPO Done Right), is equivalent to the RLOO estimation (which uses the average reward of other samples as its baseline) up to a constant scaling factor (which normally does not matter due to implementation details to normalize the advantage).[18]

The chain of equalities above is his eq. 61. GRPO proper, his eq. 58, is the same numerator divided by the group's standard deviation, which brings us to normalization.

7.4 Normalization (whitening)

Within each batch, subtract the mean advantage and divide by its standard deviation. Subtracting the batch mean is a constant baseline, so by the theorem it is nearly harmless to bias. Dividing by the standard deviation rescales every gradient in the batch by the same factor, which is an adaptive learning rate. Neither changes the expected direction of the update, and together they make Adam dramatically more stable. Add a small ϵ to the denominator.

The advantage computation for GRPO has trade-offs in its biases. The normalization by standard deviation rewards questions in a batch that have a low variation in answer correctness. For questions with either nearly all correct or all incorrect answers, the standard deviation will be lower and the advantage will be higher. Liu et al. 2025 proposes removing the standard deviation term given this bias, but this comes at the cost of down-weighting questions that were all incorrect with a few correct answers, which could be seen as valuable learning signal for the model. Those high-variance prompts can be exactly the hardest cases, where only a few sampled completions find the correct answer and provide a strong training signal.[19]

Your batch of ten episodes has the same property: a batch where nine episodes score 500 and one scores 480 will have its tiny differences inflated to unit variance. His PPO example (§6.3.5) does exactly `(advantages - mean) / (std + 1e-8)` and calls it optional but stable. That is the version to implement; whether to drop the standard deviation is a legitimate experiment once both runs work.

7.5 Why your discount factor is not 1

This framing distinction also explains why the discount factor γ is set to 1.0 in virtually all RLHF implementations. In standard RL, discounting ($\gamma < 1$) is essential: it balances the optimization between short-term and long-term reward across a multi-step episode, which is crucial for the

[18] Lambert, §6.2.8.
[19] Lambert, §6.2.8, on eq. 58.

agent to learn effective behavior over time. But in the RLHF setting, even when using the token-level MDP view, the inductive bias of the optimization is the quality of the collective completion – the reward signal scores the entire response, not individual tokens. Discounting earlier tokens would arbitrarily down-weight their contribution with no principled justification. As agentic RL settings mature – where models take real multi-step actions such as tool calls, code execution, and web browsing – discounting may become relevant again, since these involve genuinely distinct sequential decisions whose long-term consequences differ.[20]

CartPole is his "standard RL": his Table 1 in §3.1.3 lays the two settings side by side, and you are running column one, top to bottom.

8 Problem 2: reward-to-go and two baselines

Build. Make the weight computation configurable. Reward-to-go: each step gets G_t, discounted. Baseline, one of two: `rloo`, the leave-one-out advantage over the batch's episode returns, broadcast to every step of its episode (bandit framing, no value net); or `value`, a learned $V_\phi(s)$ fit by regression to G_t, with $A_t = G_t - V_\phi(s_t)$ per step (MDP framing). Normalization per batch. All three switchable by configuration so the tests can select them.

Givens. Value net same shape as the policy, one output. Adam for both, learning rate 10^{-2}. Batch of 10 episodes. $\gamma = 0.99$. 3000 episodes, seed 0.

Acceptance. Two runs. RLOO with normalization, and reward-to-go with the value baseline and normalization. Each: some window of 100 consecutive episodes averages at least 475, CartPole-v1's registered reward threshold over the conventional 100-episode window. Observe without asserting: both curves visibly less jagged than Day 1; the value loss falling as V_ϕ learns; and the two baselines reaching 475 by different routes, RLOO usually faster early, the value baseline usually steadier late.

9 Debugging signatures, Day 2

- **Reward-to-go tests pass but learning is worse.** Weights misaligned with steps. The G_t vector runs *forward* in time; its last element is the last reward alone.
- **RLOO advantages do not sum to zero.** You subtracted the full mean, not the leave-one-out mean. The tests distinguish them.
- **Value loss does not decrease.** Shape. A $[N, 1]$ output minus $[N]$ targets broadcasts silently to $[N, N]$. Flatten first.
- **Value loss falls but returns collapse.** Policy-loss gradients are leaking into ϕ. Detach the advantage before it becomes a weight.
- **Normalization explodes on a batch of identical returns.** Zero standard deviation. That ϵ.
- **Hits 500, then oscillates.** Learning rate high for a near-deterministic policy. Drop the policy rate to 3×10^{-3}; leave the value net at 10^{-2}.

---

[20] Lambert, §6.3.2.2.

10 Tests

Save as `pg_tests.py` beside `pg.py`; run `python -m pytest pg_tests.py -v`. Use `-k "not learns"` to skip the training checks, which take minutes. The tests fix the interface below and nothing else.

Interface contract.

- `make_policy(obs_dim, n_actions)`: an `nn.Module`; given observations of shape $[N, \text{obs\_dim}]$ it returns logits of shape $[N, \text{n\_actions}]$.
- `reward_to_go(rewards, gamma)`: list of T floats in, tensor $[T]$ out.
- `pg_loss(logits, actions, weights)`: scalar tensor. `actions` $[N]$ integer, `weights` $[N]$ float.
- `rloo_advantages(returns)`: tensor $[K]$ in, tensor $[K]$ out. Day 2.
- `make_value_net(obs_dim)`: an `nn.Module` whose output flattens to $[N]$. Day 2.
- `train(cfg)`: nested configuration dictionary in, list of per-episode undiscounted returns out. Layout at the top of the test file; `cfg["variance"]["baseline"]` is `None`, `"rloo"`, or `"value"`.

On the training tests. REINFORCE is seed-sensitive. A training test that fails at seed 0 but passes on two of three seeds passes; the flakiness is Day 2's lesson made visible. The training tests were desk-checked against the theory, not executed. The unit tests check arithmetic and the identities directly, including Lambert's eq. 61.

```
"""
Tests for the two-day policy-gradient briefing.
Written by Claude. Everything in pg.py is yours.

Run:  python -m pytest pg_tests.py -v
      python -m pytest pg_tests.py -v -k "not learns"   (skip the slow training checks)

Interface contract (the only thing the tests assume about pg.py):

    make_policy(obs_dim: int, n_actions: int) -> torch.nn.Module
        module(obs: Tensor[N, obs_dim]) -> logits: Tensor[N, n_actions]

    reward_to_go(rewards: list[float], gamma: float) -> Tensor[T]

    pg_loss(logits: Tensor[N, A], actions: Tensor[N], weights: Tensor[N]) -> Tensor (
    scalar)

    rloo_advantages(returns: Tensor[K]) -> Tensor[K]              (day 2)
        leave-one-out advantage for each of K episode returns

    make_value_net(obs_dim: int) -> torch.nn.Module            (day 2)
        module(obs: Tensor[N, obs_dim]) -> Tensor[N] or Tensor[N, 1]

    train(cfg: dict) -> list[float]
        per-episode undiscounted returns, one entry per episode
"""
import math
import torch

from pg import make_policy, reward_to_go, pg_loss, train
```

```
OBS_DIM, N_ACTIONS = 4, 2


def _cfg(n_episodes=1500, **variance):
    return {
        "env": {"id": "CartPole-v1", "seed": 0},
        "train": {"n_episodes": n_episodes, "episodes_per_update": 10,
                  "gamma": 0.99, "lr": 1e-2},
        "variance": {"reward_to_go": False, "baseline": None,
                     "normalize": False, **variance},
    }


# ------------------------------------------------------------------ returns

def test_reward_to_go_undiscounted():
    g = reward_to_go([1.0, 1.0, 1.0], gamma=1.0)
    assert torch.allclose(g, torch.tensor([3.0, 2.0, 1.0]))


def test_reward_to_go_discounted():
    g = reward_to_go([1.0, 1.0, 1.0], gamma=0.5)
    assert torch.allclose(g, torch.tensor([1.75, 1.5, 1.0]))


def test_reward_to_go_general():
    g = reward_to_go([2.0, -1.0, 5.0], gamma=0.9)
    assert math.isclose(g[2].item(), 5.0)
    assert math.isclose(g[1].item(), -1.0 + 0.9 * 5.0)
    assert math.isclose(g[0].item(), 2.0 + 0.9 * (-1.0 + 0.9 * 5.0))


def test_reward_to_go_shape():
    g = reward_to_go([1.0] * 37, gamma=0.99)
    assert g.shape == (37,)


# ------------------------------------------------------------------ policy

def test_policy_outputs_logits_of_right_shape():
    torch.manual_seed(0)
    pi = make_policy(OBS_DIM, N_ACTIONS)
    logits = pi(torch.randn(16, OBS_DIM))
    assert logits.shape == (16, N_ACTIONS)
    assert torch.isfinite(logits).all()


def test_policy_softmax_is_distribution():
    torch.manual_seed(0)
    pi = make_policy(OBS_DIM, N_ACTIONS)
    probs = torch.softmax(pi(torch.randn(16, OBS_DIM)), dim=-1)
    assert torch.allclose(probs.sum(-1), torch.ones(16), atol=1e-5)
    assert (probs >= 0).all()


# ------------------------------------------------------------------ loss

def _prob_of(pi, obs, action):
```

```
    return torch.softmax(pi(obs), -1)[0, action].item()


def test_positive_weight_raises_action_probability():
    torch.manual_seed(0)
    pi = make_policy(OBS_DIM, N_ACTIONS)
    opt = torch.optim.SGD(pi.parameters(), lr=0.1)
    obs = torch.randn(1, OBS_DIM)
    before = _prob_of(pi, obs, 1)
    for _ in range(20):
        opt.zero_grad()
        pg_loss(pi(obs), torch.tensor([1]), torch.tensor([1.0])).backward()
        opt.step()
    assert _prob_of(pi, obs, 1) > before, "positive weight must push P(action) up"


def test_negative_weight_lowers_action_probability():
    torch.manual_seed(0)
    pi = make_policy(OBS_DIM, N_ACTIONS)
    opt = torch.optim.SGD(pi.parameters(), lr=0.1)
    obs = torch.randn(1, OBS_DIM)
    before = _prob_of(pi, obs, 1)
    for _ in range(20):
        opt.zero_grad()
        pg_loss(pi(obs), torch.tensor([1]), torch.tensor([-1.0])).backward()
        opt.step()
    assert _prob_of(pi, obs, 1) < before, "negative weight must push P(action) down"


def test_loss_is_scalar_and_differentiable():
    torch.manual_seed(0)
    pi = make_policy(OBS_DIM, N_ACTIONS)
    obs = torch.randn(8, OBS_DIM)
    loss = pg_loss(pi(obs), torch.randint(0, N_ACTIONS, (8,)), torch.randn(8))
    assert loss.dim() == 0
    loss.backward()
    assert all(p.grad is not None for p in pi.parameters())


def test_weights_are_treated_as_constants():
    # gradient must not flow into the weights; they are data, not parameters
    torch.manual_seed(0)
    pi = make_policy(OBS_DIM, N_ACTIONS)
    obs = torch.randn(8, OBS_DIM)
    w = torch.randn(8, requires_grad=True)
    pg_loss(pi(obs), torch.randint(0, N_ACTIONS, (8,)), w).backward()
    assert w.grad is None or torch.allclose(w.grad, torch.zeros_like(w))


# ---------------------------------------------- the identity behind baselines

def test_score_function_has_zero_expectation():
    # sum_a pi(a|s) grad log pi(a|s) = grad sum_a pi(a|s) = grad 1 = 0
    torch.manual_seed(0)
    pi = make_policy(OBS_DIM, N_ACTIONS)
    obs = torch.randn(1, OBS_DIM)
    logits = pi(obs)
    dist = torch.distributions.Categorical(logits=logits)
    total = None
```

```
    for a in range(N_ACTIONS):
        pi.zero_grad(set_to_none=True)
        dist.log_prob(torch.tensor([a])).backward(retain_graph=True)
        g = torch.cat([p.grad.flatten().clone() for p in pi.parameters()])
        term = dist.probs[0, a].detach() * g
        total = term if total is None else total + term
    assert total.abs().max().item() < 1e-5


# ------------------------------------------------------------ day 2: RLOO

def test_rloo_advantage_values():
    # Lambert eq. (48)-(49): b_i = mean of the OTHER returns
    from pg import rloo_advantages
    a = rloo_advantages(torch.tensor([3.0, 1.0, 2.0]))
    assert torch.allclose(a, torch.tensor([1.5, -1.5, 0.0]))


def test_rloo_advantages_sum_to_zero():
    from pg import rloo_advantages
    torch.manual_seed(0)
    a = rloo_advantages(torch.rand(10) * 500)
    assert abs(a.sum().item()) < 1e-3


def test_rloo_equals_scaled_mean_baseline():
    # Lambert eq. (61): RLOO == (K/(K-1)) * (R_i - mean(all R))   [Dr. GRPO]
    from pg import rloo_advantages
    torch.manual_seed(1)
    r = torch.rand(7) * 100
    K = r.numel()
    dr_grpo = (K / (K - 1)) * (r - r.mean())
    assert torch.allclose(rloo_advantages(r), dr_grpo, atol=1e-4)


def test_rloo_two_episodes_is_half_the_difference_doubled():
    # with K=2 each episode's baseline is just the other's return
    from pg import rloo_advantages
    a = rloo_advantages(torch.tensor([10.0, 4.0]))
    assert torch.allclose(a, torch.tensor([6.0, -6.0]))


# ------------------------------------------------------ day 2: value baseline

def test_value_net_fits_a_constant_target():
    from pg import make_value_net
    torch.manual_seed(0)
    v = make_value_net(OBS_DIM)
    opt = torch.optim.Adam(v.parameters(), lr=1e-2)
    obs = torch.randn(64, OBS_DIM)
    target = torch.full((64,), 7.0)
    for _ in range(300):
        opt.zero_grad()
        pred = v(obs).reshape(-1)
        assert pred.shape == (64,), "value net output must flatten to [N]"
        loss = ((pred - target) ** 2).mean()
        loss.backward()
        opt.step()
    assert loss.item() < 0.1
```

```
# ---------------------------------------------------- learning (slow, ~mins)

def _best_window_mean(xs, k):
    return max(sum(xs[i:i + k]) / k for i in range(len(xs) - k + 1))


def test_learns_cartpole_day1():
    returns = train(_cfg())
    assert len(returns) == 1500
    assert _best_window_mean(returns, 20) >= 195, \
        "vanilla REINFORCE should reach the CartPole-v0 bar (195 over 20 episodes)"


def test_learns_cartpole_day2_rloo():
    # bandit-style: whole-episode return, leave-one-out baseline, no value net
    returns = train(_cfg(n_episodes=3000, reward_to_go=False, baseline="rloo",
    normalize=True))
    assert _best_window_mean(returns, 100) >= 475, \
        "RLOO + normalization should hit the v1 bar (475 over 100)"


def test_learns_cartpole_day2_value():
    # MDP-style: reward-to-go, learned V(s_t) baseline
    returns = train(_cfg(n_episodes=3000, reward_to_go=True, baseline="value",
    normalize=True))
    assert _best_window_mean(returns, 100) >= 475, \
        "reward-to-go + value baseline + normalization should hit the v1 bar (475 over
    100)"
```

Where this leaves you

You will have implemented the first three rows of Lambert's taxonomy of Ψ_t and two of the six algorithms in his Table 3.

Each algorithm in this chapter shares the same core gradient shape, but differs in how it estimates the advantage and controls the optimization: REINFORCE: The simplest policy gradient implementation, using Monte-Carlo estimates of reward and a state-based baseline to reduce variance. RLOO: REINFORCE with multiple samples per prompt, with each sample's baseline being the average reward of the others (leave-one-out) to reduce gradient variance. PPO: Adds a learned value function and a clipped policy ratio to get more accurate and stable gradient updates. GRPO: A simplified variant of PPO that groups multiple completions per prompt and normalizes rewards within the group to compute advantages, removing the need for a value function. [...] Only PPO requires a learned value function. REINFORCE and RLOO have no importance-sampling ratio – the remaining algorithms each introduce one to enable multiple gradient steps per batch of rollouts.[21]

Read the rest of that section now and each remaining algorithm is a delta on what you built. PPO (§6.2.5–6.2.7) is your value-baseline run plus an importance ratio $\pi_\theta/\pi_{\theta_{\text{old}}}$ that lets you take several gradient steps per batch, clipped to a trust region so the policy cannot move too far; his §6.2.6 case analysis is worth an hour. GRPO (§6.2.8) is your RLOO run with the group standard deviation in the denominator and the clipping bolted on. GSPO and CISPO change where the

[21] Lambert, §6.2.11.

ratio is computed and what gets clipped. Every one of them is `-(log_prob * weight).mean()` with a different weight, which is the sentence this week was for.

The one structural difference between your CartPole and his language models is his Table 1: your episodes are genuinely sequential decisions with dynamics between them, so reward-to-go and $\gamma < 1$ earn their keep; his are single-step bandits over whole completions, so RLOO and GRPO dominate and $\gamma = 1$. Having built both framings, you can read either literature.

---

11 Appendix: how this document was verified

Every equation and every claim of "unbiased" or "biased" above was checked by `verify_theory.py`, shipped alongside this PDF. Run it with `python verify_theory.py`; it needs only `numpy` and `sympy`. Thirty-one checks, three kinds.

Symbolic. With `sympy`: the log-derivative trick; the zero-mean identity for a softmax policy, for every parameter; the baseline theorem in the same setting; the RLOO identity of Lambert's eq. 61 as an exact symbolic equality for $K = 2, \ldots, 8$; that RLOO advantages sum to zero; and both reward-to-go facts, the recursion and the γ^t footnote.

Exact enumeration. A two-state, two-action, horizon-three MDP with stochastic transitions and a tabular softmax policy is small enough that every trajectory can be listed with its probability. Each expectation in the briefing is then a finite weighted sum, and "equals $\nabla_\theta J$" is checked against a finite-difference gradient of the exactly computed objective, to 10^{-6}. Confirmed at both $\gamma = 1$ and $\gamma = 0.9$: the Day 1 estimator; strict reward-to-go with the γ^t; the value-function baseline; an arbitrary state-dependent baseline. Confirmed as negative controls, which matter as much: dropping the γ^t is exact at $\gamma = 1$ and *biased* at $\gamma = 0.9$; an action-dependent baseline is biased. The variance ordering is measured exactly, not sampled: full return $>$ reward-to-go $>$ reward-to-go with a V baseline, at both discounts. RLOO with $K = 2$ is checked over all pairs of trajectories, and mean-baselining is confirmed to give $\frac{K-1}{K}\nabla_\theta J$, the scaling of eq. 61 seen from the other side.

Fuzzing. Reference implementations of the two arithmetic functions the tests exercise (reward-to-go and the RLOO advantage), checked against brute force on 500 random inputs each, plus every fixed vector the test file asserts. The loss-sign test is replayed in `numpy` with the analytic softmax gradient on 200 random cases. Whitening is checked to be an invertible affine map.

Environment facts. Checked against the installed `gymnasium`: CartPole-v1 has a 500-step limit and a registered reward threshold of 475; v0 has 200 and 195; the pole threshold is 12° and the cart threshold ±2.4; `step` returns a five-tuple; reward is +1 per step.

What is not verified. The two training tests. They depend on your implementation and on REINFORCE's seed sensitivity, and no `torch` was available where this was built. Their thresholds are the intent; treat two-of-three seeds as passing. A Lean proof was requested and is not included: no Lean toolchain was available, and the identities here are finite sums over a softmax, which `sympy` settles exactly and a Lean-with-Mathlib port would restate rather than strengthen.